

Realizing Self-Regulating Cars on Commodity Hardware

Anonymous Authors¹

Abstract—A free-flow road network can devolve into a traffic jam with a local disturbance that pushes the density past a critical point, a phenomenon known as a sudden traffic jam. This instability in traffic flow has been studied in the past with different infrastructure and vehicle based control approaches. One notable approach called self-regulating cars learned a controller that prevents this by setting a desired speed per road segment using a central cloud observer. In this paper, we ask what is the minimal augmentation needed to an ordinary vehicle for it to become self-regulating in a decentralized manner.

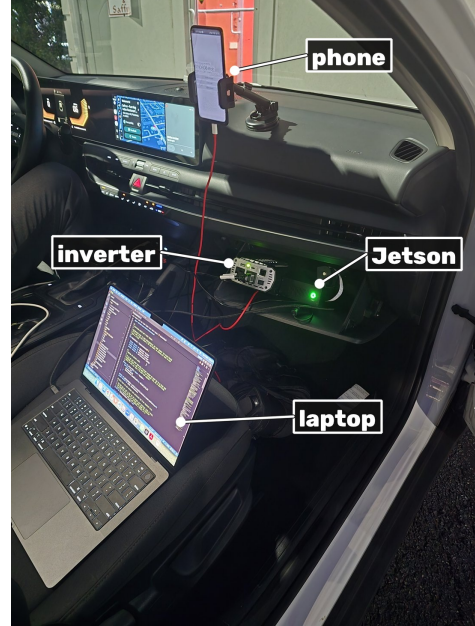
Our answer has three parts: a phone the driver already owns (OnePlus Nord N10), an onboard computer (NVIDIA Jetson Orin Nano- US\$250), and a public traffic API (HERE free tier). Nothing is installed on the road, and no server sits inside the control loop. The policy is a deterministic function of a feed every vehicle can query, so vehicles carrying the same weights compute the same action without coordinating. The phone samples its camera, GPS and inertial unit at rates the Jetson sets at run time rather than at install time.

Eight drives on one day recorded 22,929 ticks over 152.8 minutes on vehicle power. The on-Jetson path has a median of 19.8ms and the phone-to-Jetson path a 95th percentile of 116.19ms against a 200ms target. Nothing was connected to the vehicle’s controls, and no advisory was acted on by a driver. In simulation on a prior-tested network, the deployed configuration gains 230 ± 44 veh/h over no control while the published self-regulating cars configuration gains 224 ± 61 veh/h, establishing that our decentralized deployment is as performant as the theoretical centralized controller. Two deployment failures are reported because a simulator cannot produce them: the phone overheated on a windscreen mount in sunlight and the android platform stalled the pipeline, and a ninety-degree camera rotation sent detection to a null during the drive.

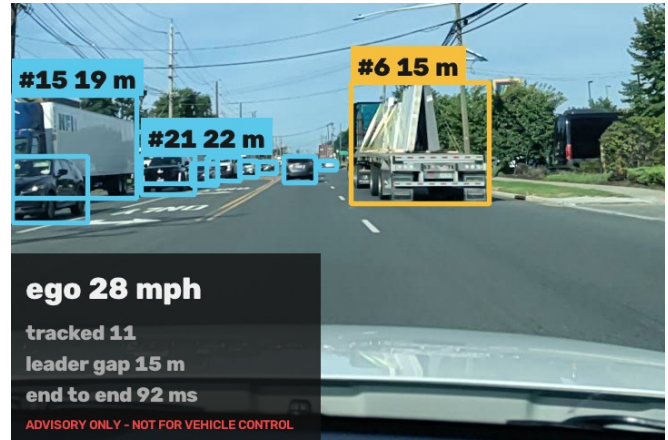
I. INTRODUCTION

A free-flow road network can jam without any apparent cause, without a crash or a fixed bottleneck. A small burst of inflow traffic can push the local density past a critical value and the queue then feeds itself until the demand drains away [1], [2]. This phenomenon in the literature is coined as a sudden traffic jam. Several prior studies have focussed on mitigating such traffic flow instabilities. For example, learned controllers that smooth mixed-autonomy traffic report gains at low penetration in microscopic simulators [3], [4], [5].

Another work in this space introduced the concept of self-regulating cars [6]. [1] notes that the fundamental diagram of traffic flow provides an explanation for the underlying process of sudden traffic jams. The flow rises with density to a critical point and falls after it [7], [8], [9]. Therefore, a controller that holds density below that point protects throughput, and it does not need to know anything about the rest of the network to do so.



(a) Everything added to the vehicle.



(b) What it computes, on a recorded drive frame.

Fig. 1. The deployment prototype. The phone supplies the camera, GPS and inertial unit and carries the display; the Jetson runs perception, the policy and the safety gate; the inverter is the only connection to the car. The laptop is for debugging and is not part of the rig. Nothing reaches the throttle, the brakes, the steering or the vehicle bus. In (b), we show the camera perception module. The leader in the ego lane is amber and the other tracked vehicles are cyan.

Earlier work introducing self-regulating cars built such a controller and measured it. That work divided the Mainz highway network into twelve super-segments, learned a policy that assigns each super-segment a desired speed, and applied that speed to the vehicles on it. In PTV Vissim

¹Anonymous Affiliations. anonymous.emails

TABLE I

WHAT A VEHICLE HAS TO ACQUIRE. NO PART OF THIS SYSTEM IS INSTALLED ON THE ROAD, AND NO PART OF IT IS WIRED INTO THE VEHICLE.

Part	Supplies	Cost
Jetson Orin Nano developer kit	perception, policy,	US\$250
Android phone	safety gate camera, GPS, inertial unit, display, cellular	US\$150
Mount, 12 V in-verter, cable	power and mounting	US\$100

the policy raised throughput by 5% and cut average delay by 13% against no control. The result holds at partial adoption and improves as the controlled fraction grows. Its orchestration model is centralized. A central coordinator collects the network state, computes the action, and sends every participating vehicle its desired speed.

While the results shown by [6] and others are promising, they remain limited to simulations. Deployment, an extremely important and difficult step for impact, is generally treated as an engineering detail rather than as a research question. What it would cost goes unreported, except for a few exceptions [10], [11].

This paper tackles deploying self-regulating cars as its central question. Ease of adoption is decided by the size of the change, because the controller’s benefit grows with the fraction of vehicles carrying it. That fraction is therefore the quantity a deployment design has to maximise, and anything raising the cost of one installation lowers it. Judged this way, two properties of the centralized model are expensive. The first is the observation: the policy reads six features per super-segment, and four of them are vehicle-level ground truth a vehicle cannot easily obtain. The second is the cloud coordinator, which has to be built, paid for, kept available and trusted, and which sits inside the control loop. Other designs put a floor under the same cost elsewhere, whether through a purpose-built vehicle [11], a roadside sensor [12], [13] or a per-vehicle service contract [6]. The target here is the opposite: the least that has to be added to a vehicle that already exists. We call this the minimum viable deployment and discuss it in Section II. Our proposed solution is summarized in Table I. The computer is an NVIDIA Jetson Orin Nano developer kit, listing at about US\$250 [14], and the phone used for the drives reported here is a OnePlus Nord N10 (2021 model), a mid-range handset rather than a flagship. Figure 1 is the prototype we built, installed and running.

The division of labour inside the rig is as follows: the phone captures and forwards, the Jetson interprets and decides. Nothing is wired to the throttle, the brakes, the steering or the vehicle bus. The only connection to the vehicle is a 12 V power lead, so the installation changes no system the vehicle already depends on. The advisory is shown to the driver using the phone display, and they decide their level of compliance. That is an arrangement a navigation

application already runs. Mobile telematics platforms operate the same driver-facing rail at the scale of tens of millions of drivers [15], [16]. This human-in-the-loop design supports our claim about deployability, namely that the interface needs no new hardware, no new habit, and no regulatory change.

Scope of the work. No advisory this project computed has ever been acted on. On every drive, the human drove without interfere while the Jetson rendered a recommended speed on the phone’s display. There are two independent reasons for this. An experiment in which an advisory influences a human driver requires approval from an institutional review board, which this project does not hold. One instrumented vehicle also cannot produce a flow effect, so a compliance experiment on a single car would establish nothing about throughput or delay. In this paper “deployed” means that the system ran on a road and computed its outputs there. It does not mean that the recommendations were acted on. The flow-level claim is made in simulation for the same reason.

Contributions.

- A vehicle-side realization of the self-regulating-cars deployed using commodity parts, with a sensing scheduler that sets every sensor’s sampling rate at run time (Sections II and III).
- A measured account of running the system on a road: eight drives, 152.8 minutes on vehicle power, with per-stage timing and field provenance, and the deployment experiences that were produced. The phone overheated from its own load and from sunlight on the windscreen, and a ninety-degree camera rotation sent detection to zero (Section IV).
- A flow-level measurement of the observation change. On a SUMO port of the same Mainz network, a policy restricted to the five fields a traffic API returns gains 230 ± 44 veh/h over no control, and the original six-feature policy gains 224 ± 61 veh/h (Section V).

II. THE MINIMUM VIABLE DEPLOYMENT

This section states the minimum a vehicle has to acquire for the self-regulating cars vision to be deployed on it, and Figure 2 provides the overview of the system design.

Two sensing layers, and two observation spaces. The deployment carries two decisions, and we give each its own sensor. The division is as follows: the advisory layer reads a traffic API per super-segment and decides a desired speed, and the safety layer reads the camera, GPS and inertial unit and decides whether that speed may be shown. The reason for the split is that neither sensor can supply the other layer’s inputs. A traffic API reports aggregate quantities per road segment and contains no individual vehicles, so it cannot report a leader distance or a follower gap. A forward camera is the opposite, as it measures the vehicle ahead and reports nothing beyond its own horizon. Therefore, each decision is built on the only sensor that can supply it.

The advisory observation space. The advisory layer reads the whole network at once, as a fixed partition into super-segments with five fields each. Three of the five come from the traffic feed, which in this deployment is the HERE Traffic

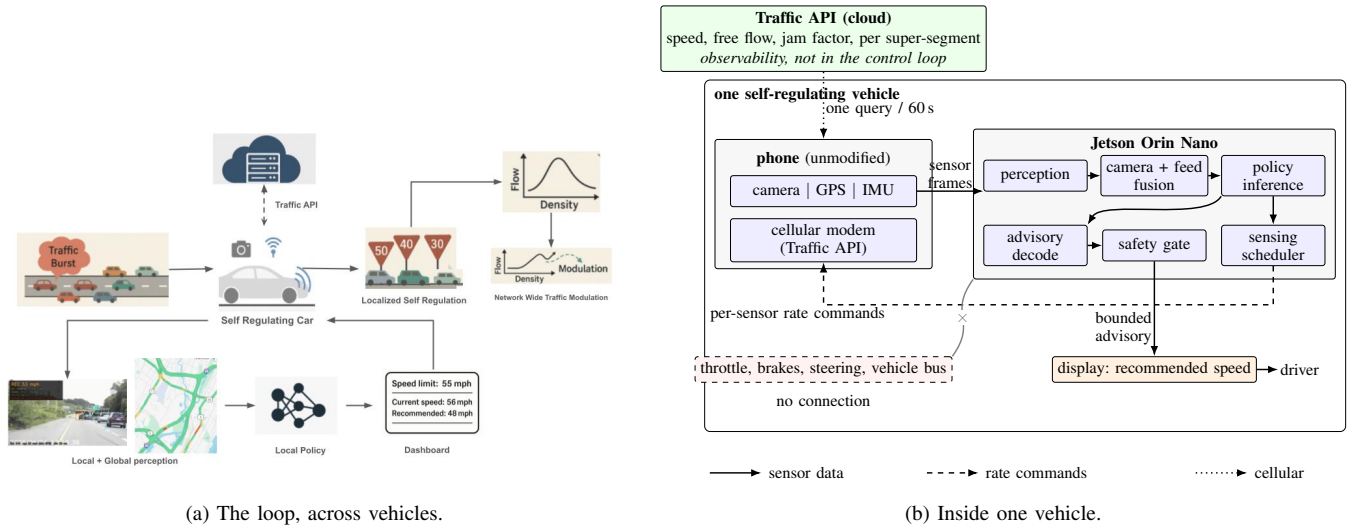


Fig. 2. The deployment, at two scales. In (a) a burst of inflow pushes a stretch of road past its critical density; each self-regulating car reads the network from a traffic API and the road ahead from its own camera, runs the policy locally, and shows the driver a recommended speed. The aggregate of those local actions is what moves the network along its flow-density curve, and no vehicle talks to another vehicle. In (b), we detail one of those vehicles. The phone captures or queries at the rate it is told and forwards raw data, and every interpretation and sampling decision is on the Jetson. The dashed box is the set of vehicle systems that are not touched in this design.

API [17], and two come from the map. They are as follows. `speed` is the segment’s current aggregate speed, which is the quantity being regulated, and `free_flow` is the speed it carries when uncongested, so the two together state how far below its own capacity the segment is running. `jam_factor` is a congestion index on a 0 to 10 scale, which summarises the same shortfall on a bounded scale the feed publishes directly. `lanes` turns a per-segment reading into a per-lane one, and `length.km` says how much road that reading describes. The feed also publishes `confidence` and `traversability`, and we read neither. This is because the policy is trained in the SUMO simulator and neither can be expressed as an observation in it. Also, they would not add much, as across the 915 flow records logged in our drives, confidence is at or above 0.97 on 729 of the 909 that carry it, and traversability is a binary field that reports closed on 6, where `jam_factor` has already saturated at 10.

The action is one of three speed fractions per super-segment. The base it multiplies is that segment’s static speed limit from the map, and we deliberately do not use the feed’s free-flow reading for it, because a live reading used for the base as well as for the observation would enter the loop twice. The driver sees one number, which is the row for the super-segment the vehicle’s position currently falls on.

The safety observation space. The safety layer reads the road immediately around the vehicle, and its inputs come from three sources. The forward camera supplies the distance to the vehicle ahead and the rate at which that distance is closing. GPS supplies ego speed, and the slope of that speed over a short window supplies ego acceleration. The tracked vehicles in range supply a local density and a local mean speed. Therefore, the layer can judge whether a recommended speed is unsafe or obstructive at this moment, which is a judgement about the road inside the camera’s

range rather than about the network.

Why not aggregate from local sensing. Another design could reconstruct segment-level state from a vehicle’s own sensors and those of its neighbours. While that removes the dependence on a traffic feed, it needs either vehicle-to-vehicle exchange or a server that aggregates, and the second is exactly the deployment cost we want to avoid. Local sensing also needs a sufficient number of compliant vehicles before it can produce a rich segment state, so it cannot be tested with a unit deployment at all. Therefore, we keep the per-vehicle quantities where they are measurable, and they feed the safety layer only.

Decentralized execution. The policy’s input is the whole network’s state, and the policy is a deterministic argmax over three speed fractions. Every vehicle holding the same weights, reading the same map and querying the same feed computes the same action for the same super-segment. Therefore, decentralized execution reproduces centralized execution by construction. What differs on a road is staleness, because a response carries the time it was received, which this system measures, and not the time at which the conditions it describes held. The query itself is small, at $O(100)$ numbers per decision on a 60 s interval, and one request by corridor or circle covers the whole extent.

What the vehicle must therefore carry. The list has five entries. (1) A network definition that names the super-segments and their edges. (2) A traffic-feed query bounded by that network’s extent. (3) A policy trained on that network. (4) A local sensor set for the safety layer. (5) A display. None of the five connects to the throttle, the brakes, the steering or the vehicle bus, and none of them requires a server inside the control loop.

III. THE SYSTEM AS BUILT

The rig. The vehicle side is an ordinary Android phone and a Jetson Orin Nano, with no purpose-built hardware (Figure 2(b)). The phone holds no interpretation logic, as it captures or queries at the rate it is told and forwards raw data. Every association, fusion and sampling decision happens on the Jetson. The phone reaches the internet over 5G. We implement two transports for the same wire protocol, a Tailscale network for development with the phone in hand, and USB in the vehicle. Both devices are powered from the vehicle, so energy is not a binding constraint.

The sensing scheduler. Each sensor’s sampling rate is set at run time by the Jetson and commanded to the phone. The four rates are camera, GPS, inertial unit and traffic feed, and the scheduler trades them against three costs: traffic-API quota, thermal headroom on the phone, and Jetson compute. It has two modes. In shadow mode the decision is recorded and not sent, and in live mode the phone applies it. That distinction is about sensing configuration only. Therefore, the word “live” here describes what the sensing configuration did, and not what the vehicle did.

The transport. One wire protocol carries everything between the phone and the Jetson, over the development link and the in-vehicle link alike. It defines eight channels, each with its own sequence numbers, priority, overflow policy and queue depth, over a shared timebase, and we freeze the frame layout in a single specification file. Neither implementation is tested against the other, and both are tested against that file. Therefore, the two sides cannot drift by agreeing with each other.

The safety gate. The advisory passes a filter before it is displayed. Because the advisory is a speed, a rule earns its place only by bounding that speed. For this, we have two rules. The first raises a recommendation that would have the vehicle travel slowly on an uncongested road, which is the obstruction case. The second overrides the recommendation when the gap to the vehicle ahead, or the time left to close it, falls below the configured minimum. Each rule records per tick whether it fired, stayed quiet, or could not be evaluated, naming the missing inputs. A rule that cannot be evaluated is inert, and does not change the recommendation.

The device cannot drift from what was trained. We vendor the observation and action contract onto the device and pin it to a named version of the simulator, so the device encodes an observation and decodes an action exactly as training did, and the export step refuses any dimension mismatch. A second frozen specification file holds the encoded observations, the action schema, the decoders and the neutral fallbacks. We derive every entry twice, once from the pinned simulator and once from the vendored copy, and the generator refuses to write the file on any disagreement. The check that runs on the device needs that file and nothing else. Therefore, the property is testable on the hardware as well.

Two design stances. We take two stances in our system’s design. The first is to degrade rather than stop, because debugging time inside a vehicle is expensive: missing GPS,

TABLE II

THE WHOLE ROAD CORPUS. ONE DAY, ONE VEHICLE, EIGHT RUNS.

runs	8: six shadow (17,948 ticks), two live (4,981 ticks)
ticks	22,929, GPS fix valid on 22,734 (99.1%)
recorded span	152.8 min, of which 114.3 min above walking pace
distance	between 86.55 and 90.06 km
video	1.676 GB, 19,000 indexed frames across six runs
traffic feed	123 response bodies, 3.7 MB, every one HTTP 200
inertial archive	658,862 raw samples in 783,519 records 187 files, 2,180,405,759 bytes, verified by digest against the device

no camera, no display and no trained checkpoint each have a defined degraded mode. The second is that the hot path holds no queues and the latest value wins, so a frame that was never processed is dropped, a slow display skips ticks, and a stalled card drops log records. For a real-time advisory, stale data is worse than missing data.

Data collection. A tick is one pass of the Jetson’s pipeline. It takes the most recent camera frame with the latest GPS and inertial readings, runs detection, tracking and distance estimation, builds the observation, runs the policy, decodes the advisory, passes it through the safety gate, and writes one record. Camera frames pace the ticks, so the tick rate follows the commanded camera rate. Every per-tick count in this paper refers to that record.

IV. RUNNING IT ON A ROAD

The system was driven on 2026-09-08 in Westfield, New Jersey, on a OnePlus Nord N10, and Table II is the whole corpus. We report distance as a bracket rather than a number, because the two available methods disagree. Summing great-circle steps between fixes less than 3 s apart gives 86.55 km and omits distance covered during gaps, while integrating the reported speed gives 90.06 km and counts the gaps but trusts the speed field. Therefore, quoting either figure alone would overstate the precision.

The shakedown drive. The first drive ran 1,632 ticks over 279.5 s and 6.24 km, at a mean speed of 22.9 m/s. The mean tick rate was 4.69 Hz with a median spacing of 203 ms, and two gaps longer than 2 s totalled 5 s. Distance agrees to 0.01 km between integrating the reported speed and the great-circle path through the fixes, which is two independent routes to one number. That drive also produced three defects, as follows: the camera frames arrived rotated by ninety degrees, the frames were being discarded and not logged, and the severe thermal tier ended a session rather than lowering rates. We fixed the second at the roadside with a laptop attached to the Jetson, and the other two are the subject of the last two parts of this section.

Latency. Table III gives the two measurements. The on-Jetson path is a bench measurement on pre-stamped frames, so it excludes capture wait. The phone-to-Jetson figure is the

TABLE III

MEASURED LATENCY. THE UPPER BLOCK IS THE ON-JETSON COMPUTE PATH, MEASURED ON THE DEVICE ON PRE-STAMPED FRAMES. THE LOWER BLOCK IS THE PHONE-TO-JETSON PATH, POOLED OVER THREE 180 S BENCH RUNS, 2,684 TICKS.

Stage	Measured
detection (YOLOv8n, TensorRT FP16)	17.7 ms p50, of which 3.9 ms is GPU
tracking and distance	1.1 ms p50
observation and encode	0.4 ms p50
actor and advisory decode	0.5 ms p50
on-Jetson end to end	19.8 ms p50, 20.2 ms p95, 48.5 FPS
phone to Jetson over USB	116.19 ms p95
the same path over Tailscale	215.63 ms p95
the USB wire hop itself	of the order of its own 2 ms conversion bound

pooled 95th percentile over three bench runs of 180 s each. Over USB it is 116.19 ms and over the Tailscale network it is 215.63 ms. The largest and smallest of the three USB runs differ by 48.80 ms at the 95th percentile, and that difference has one primary source. The phone-side send-queue stage differs between the same pair by 46.40 ms, while every other stage differs by less than 2 ms. Therefore, the spread is phone-side queueing rather than the link. We report the wire hop itself as an order rather than a duration on purpose. Its two endpoints are stamped on different devices and therefore on different clocks, and every conversion between them carries an error bound of about half the minimum round-trip time. Its median is 1.97, 2.16 and 2.03 ms across the three runs, against conversion bounds of 1.95, 1.86 and 2.06 ms for the same runs, so the quantity is not separable from the bound on the instrument that measured it.

Every drive reproduces its own recorded decisions. A replay tool re-runs each logged tick’s inputs through the incumbent sensing scheduler and requires the recorded decision back. All eight drives pass, with 0 of 22,929 ticks mismatched. These runs support the claim that each log reproduces the decisions it recorded.

The two clocks, and conversion between them. The phone and the Jetson each run their own clock, and the two are not synchronised, so any interval with one end on each device has to be converted. No conversion here returns a bare number. It returns the instant, how far that instant could be off, and which estimate produced it, and it fails rather than return a margin too wide to use. The margin is half the fastest round trip the two devices have exchanged plus the drift since, because a round trip cannot reveal how its time divided between the two legs. Three runs bound that machinery, as follows. A null case with both ends on one machine, where the true offset is zero by construction, stayed within 12 microseconds over 145 s. A real link over 330 s gave a fastest round trip with a median of 14.8 ms and a margin with a median of 8.0 ms across 357 exchanges. The same link under full sensor load completed 218 exchanges

with every channel still on cadence. Across these runs the two clocks drift apart at 12 parts per million.

Deployment experiences. The rest of this section reports the challenges the unit deployment produced. They are not incidental to the contribution, because they are the part of it that a simulator cannot supply.

Thermal behaviour is the binding constraint. The thermal backoff works on hardware. Across two 900 s bench runs the rule fired on 1,504 of 3,486 ticks and on 1,561 of 3,451 ticks, on a recorded cause of warm skin temperature, and the phone delivered 3.000 and 2.998 Hz against a commanded 3.0 Hz. On the first drive the handset reached the severe tier on a windscreen mount in sunlight, with one zone at 51.6 C, and the backoff correctly commanded 1.5 Hz. While the backoff behaved as designed, what ended the drive was the Android platform throttling the application and stalling the pipeline: the final tick recorded a link time of 135,359 ms against a median of 52 ms, and the application then closed the session. Nothing in the system being tested detected or reported that condition, and no gate read the link time. The failure was sharp rather than gradual. Every tick up to 1311 had a normal link time, so 279 s and 6.24 km of that drive remained usable and only the last tick was discarded.

Perception fault. The camera frames arrive rotated by ninety degrees. The detector expected an upright road scene, so on a rotated frame it returned almost nothing. The measurement that identified the cause put the same frames through the same detector twice. As delivered they returned 0 vehicles, and rotated clockwise they returned 2 at confidence 0.90 and 5 at 0.87, and only the clockwise rotation recovered them. One such frame shows a van filling much of the picture with a legible plate, so this was not a marginal detection. Figure 1(b) is a drive frame carried through the perception stage.

The vehicle is a hostile environment and the faults are physical. Both devices ran from vehicle power for 152.8 minutes across eight runs with no power interruption. Two install faults each cost drive time. One handset’s cable failed in the vehicle, so we moved the application to a second handset mid-session, with the swap recorded automatically in the run metadata. Direct sunlight on the dashboard heated the tethering handset until its hotspot shut off, which ended a drive. Overall, the only solution we had to continue the data collection was cooling the phone by putting it in shade. Since the purpose of our drive was data collection and system testing, we ran it on relatively high sensing frequency. Based on the simulation results presented in section V, setting the camera capture rate to capturing one frame per minute should mostly resolve the heat issue.

V. THE FLOW-LEVEL RESULT

One instrumented vehicle cannot demonstrate throughput or delay. Therefore, the flow-level claim comes from simulation. The question the simulation answers is narrow, namely whether restricting the controller’s observation to the fields a traffic API returns costs throughput.

The two observations being compared. The prior controller [6] reads six quantities per super-segment: density, lane count, mean speed, mean gap, entry rate and exit rate. Four of the six are vehicle-level ground truth in the published implementation and a vehicle cannot obtain them, because the gap is the mean of each vehicle’s follow distance, the entry and exit rates are set differences over the vehicle identities present one minute apart, and the density is counted over the vehicles on the segment. The deployed observation of Section II replaces those six with the five fields a feed and a map supply. We leave the reward unchanged and keep training centralized on ground-truth density. Therefore, only the state the controller reads at decision time differs between the arms.

On the road those five fields come from the HERE Traffic API [17], whose *jam factor* is a congestion index on a 0 to 10 scale. The simulator has no such feed. We read speed and free flow from the road itself, and compute the jam factor as $\text{clip}(10(1 - \text{speed}/\text{freeFlow}), 0, 10)$. While that expression reproduces the scale, HERE does not publish the formula behind its own jam factor, and it folds in incident data this project does not have. That expression is therefore a stated model rather than a measurement, and it is the one place where the simulated observation and the deployed observation are not the same quantity.

The port. The published controller [6] ran on the Mainz network in PTV Vissim under a Wiedemann-99 car-following model. We converted that network to SUMO [18] directly from the Vissim XML, producing 410 edges over 51.5 km. Three conversion decisions matter, and all three apply identically to every experiment. The source network is flat in the dynamics Vissim runs, because every one of its 410 links carries a gradient of zero, while its geometry carries vertical offsets of up to six metres that separate overpasses from the roads beneath them. The importer reads gradients off that geometry and produces edges up to 923% steep, and SUMO applies gradient to acceleration. Therefore, flattening the import restores the source’s own dynamics. Every edge imports with a 50 km/h limit while the action set tops out at 60 km/h, so a speed factor of 1.2 makes all three actions expressible. We model the exit as a three-into-two lane drop, because the source link ends into free outflow and the network otherwise has no bottleneck there. The conversion also does not import right-of-way: the port carries 2 junctions with conflicting movements against 10 active conflict areas in the source.

The fleet. The fleet is the Extended Intelligent Driver Model with a one-second reaction time [19], [20], which we chose because it has a capacity drop. A capacity drop is not a free parameter, because real queue discharge runs 5 to 20% below free-flow capacity, which is a property of traffic rather than of a model [21]. Measured on this port, the Wiedemann-99 calibration discharges 4.3% *faster* than free flow, at 1,980 veh/h against 1,899 veh/h, which no measurement of real traffic supports. The Extended Intelligent Driver Model drops 21% at an isolated lane drop. The gate we applied before any experiment was run is that served flow must fall

TABLE IV
HELD-OUT SEEDS 16 TO 30 ON THE MAINZ PORT, PAIRED ON SEED. THE ZEROED-HEAD ARM IS THE SAME NETWORK WITH ITS OUTPUT LAYER ZEROED, AND IT CARRIES THE ENTRY GATE THAT BOTH CONTROLLED ARMS CARRY.

Arm	Flow (veh/h)	Paired gain	Speed (km/h)
Traffic-API observation	3,765	$+230 \pm 44$	40.7
Published six features	3,759	$+224 \pm 61$	41.0
No control	3,535	reference	42.3
Zeroed output layer	2,847	-688 ± 46	28.8

as offered demand rises, by more than the seed spread.

Protocol. The seed split is as follows: seeds 1 to 10 train, seeds 11 to 15 select the checkpoint, and seeds 16 to 30 evaluate. Four arms across fifteen seeds is 60 episodes. Each episode runs 2,500 s at a 0.5 s step, with the measurement window opening at 900 s, under a sustained demand of 4,500 veh/h and 3,124 scheduled departures, at 100% penetration.

Why the window opens at 900 s. Served flow is zero until the first trips complete and does not reach its sustained level before then. Over 200 s windows across the fifteen evaluation seeds, the no-control arm completes 0, 43, 79 and 139 trips in the four windows before 800 s, and between 188 and 214 in every window after. Therefore, measuring from the start of the episode would average the fill into the result. The window makes the throughput figure a measurement of the controller rather than of the fill. Arms are paired on seed, and every error bar below is two standard errors of the paired difference.

The result. We present the results in Table IV. The two controlled arms differ by 6 veh/h against error bars of 44 and 61 veh/h. Therefore, restricting the observation to what a traffic API returns costs nothing, and the deployed observation is inseparable from the published one in [6]. Both controlled arms run slower than no control while serving more vehicles, as space-mean speed falls from 42.3 to 40.7 km/h and arrivals rise from 1,940 to 1,995 per episode. That is different from the findings in [6], but the difference in the simulation environment and geometry makes this unattributable. The throughput results replicate because holding density below critical upstream of the merge lets the merge discharge near capacity instead of breaking down.

Testing the boundary. Both controlled arms hold a vehicle at an entry link while that link is at or above critical density, and the no-control arm does not. This can be understood to be a replication of the self-regulating mechanism on inflow links that lead to the entry nodes in the network. That gate is a boundary condition on the controlled system rather than a piece of road, and applying it to an uncontrolled run would credit that run with a control action it is not taking. The asymmetry is a candidate explanation for the gain, so we tested it. The zeroed-head arm is the same trained network with its output layer zeroed after loading. It carries the entry gate and loses 688 ± 46 veh/h against no control, which is 15.7 times the no-control arm’s own bar below it. Therefore,

the gate alone does not produce the gain.

What this result is not comparable to. No number here is comparable to the published table in [6], and every claim is against this port's own no-control baseline. The two runs share the controller and the shape of the topology, and they differ in almost everything that decides how the controller behaves: PTV Vissim against SUMO, Wiedemann-99 against the Extended Intelligent Driver Model at a one-second reaction, and an 18,000 veh/h surge against a sustained 4,500 veh/h. Therefore, the pair is stronger evidence than a matched number in one simulator would be.

The simulated gate and the deployed gate answer different questions. In simulation the advisory is written as a speed command and SUMO bounds it by the car-following model's own safe speed, so a vehicle whose leader is slower follows its leader. That is the semantics the published implementation has and the documented way to command a speed in SUMO, and we left it alone. Replacing the simulator's car following with our own layer would make this arm incomparable to the earlier paper, and it would make the run reproducible only by someone holding our code rather than by someone holding SUMO and the network file.

On the road that bound would be the wrong instrument. SUMO's safe speed is an actuator command, and it works because the simulator sets the vehicle's speed. The prototype sets nothing. The driver does the car following, so the vehicle already carries the model the simulator supplies, and a second copy of it on the phone would recompute what the driver is already doing. The question the road poses is whether a recommendation is fit to show, and that is what the deployed filter decides. One rule raises a recommendation that would have the vehicle travel slowly on an uncongested road. The other withholds the number when the gap to the vehicle ahead, or the time left to close it, falls below the configured minimum. Therefore, neither has an analogue in the simulator, because a simulated vehicle has no display and no driver reading it.

VI. RELATED WORK

The controller this paper deploys. Self-regulating cars learn a policy that assigns a desired speed to each super-segment of a highway network, and reports a 5% throughput gain and a 13% delay reduction against no control in PTV Vissim [6]. Its observer is central and its decisions reach vehicles through a cloud service. [1] characterizes the failure mode it targets, defining a sudden traffic jam as a burst that pushes density past a critical threshold into a stable low-capacity state. This paper changes where the policy is evaluated and what it is allowed to read, and it leaves the reward, the training and the controller unchanged.

Sparse vehicles as traffic actuators. A single controlled vehicle damps stop-and-go waves on a ring track [10], which established the sparse-actuator premise. Learned controllers improve simulated traffic at low penetration in the mixed-autonomy literature [3], [4], [5]. CIRCLES placed about one hundred controlled vehicles into live interstate traffic

and is the strongest field evidence available, using purpose-built vehicles and a centralized speed planner over cellular links [11]. While those results settle the control question, the work here differs on the deployment axis rather than the control axis: the vehicle is unmodified, the compute is commodity, and no planner is contacted at run time.

Infrastructure-centric control. Ramp metering, variable speed limits and adaptive signal control remain the deployed tools [12], [13], [22], [23]. While they are effective where the sensors and actuators exist, they require owning the roadside.

Traffic state from vehicles and from feeds. Probe-vehicle systems estimate network state by uplinking per-vehicle traces to a backend that aggregates centrally [24], [25], and connected-vehicle standards broadcast per-vehicle position, speed and heading at up to ten hertz [26], [27], [28], which the collective perception service extends to per-object detections [29]. This design uses neither, because the observation it needs is already published as a commercial product at segment granularity [17]. Therefore, a query replaces both the radio and the aggregation backend. The trade is explicit: while a feed is a third-party dependency with an unreported lag and different risks, it suffices for a minimum viable deployment.

Advisory interfaces and edge perception. Coarse, human-followable advisories have precedent in advisory-autonomy work [30], [31], [32], and mobile telematics platforms operate the driver-facing rail at the scale of tens of millions of drivers [15], [16]. This paper does not evaluate compliance, for the reasons stated in Section I. On the sensing side, embedded detectors and Jetson-class hardware make on-vehicle perception practical [33], [34], [35], [36], and the measurements in Section IV are consistent with that literature. The contribution here is not the detector. It is what the rest of the system around it has to do to keep the detector's output honest when it is wrong.

VII. CONCLUSION AND LIMITATIONS

This paper asked what an ordinary vehicle must acquire to become self-regulating without a coordinator. The answer is a phone the driver already owns, a US\$250 computer and a public traffic feed, which together run a controller that previously needed a central observer and vehicle-level ground truth. The parts list is the result, and not an implementation note. The benefit of a vehicle-side controller grows with the fraction of vehicles carrying it, so the cost of one installation decides whether it ever reaches a road.

Two findings generalize beyond the controller we deployed. The first is that an observation can be queried instead of sensed, because the four quantities a vehicle cannot obtain are why the published design needed a central observer, and a commercial feed supplies a substitute at no cost to the estimated performance. The second is that decentralization then requires no protocol, because a policy that is a deterministic function of a public feed makes vehicles holding the same weights agree by construction. Therefore, the obstacle to a self-regulating fleet is the cost of the first installation and

the driver who decides whether to follow the number, rather than the control algorithm.

Limitations. The chain from a displayed number to a smoother road has one link this paper does not test. We showed that the system computes an advisory on a real road, and not that a driver follows it, for the reasons Section I gives. The flow-level claim therefore rests on simulation, on a port with its own baseline rather than a reproduction of the published one. Replacing a cloud observer with a commercial feed also exchanges one dependency for another, because a feed carries an unreported lag and terms we do not control. In future work, we aim to look at incentive structures for driver compliance, and other free tier traffic providers for reducing dependency and increasing the robustness of the system.

VIII. ACKNOWLEDGEMENTS

Opus 5 (1M) model was used for writing the code as well as the text of the paper. All sections in the paper and code was co-worked on by AI and human authors, with human verification.

REFERENCES

- [1] A. Authors, "Anonymous title," *Anonymous Publication*, Anonymous Year.
- [2] Y. Sugiyama, M. Fukui, M. Kikuchi, K. Hasebe, A. Nakayama, K. Nishinari, S.-i. Tadaki, and S. Yukawa, "Traffic jams without bottlenecks: experimental evidence for the physical mechanism of the formation of a jam," *New journal of physics*, vol. 10, no. 3, p. 033001, 2008.
- [3] C. Wu, A. Kreidieh, K. Parvate, E. Vinitzky, and A. M. Bayen, "Flow: A modular learning framework for mixed autonomy traffic," *arXiv preprint arXiv:1710.05465*, 2017.
- [4] E. Vinitzky, A. Kreidieh, L. L. Flem, N. Kheterpal, K. Jang, F. Wu, C. Wu, R. Liaw, E. Liang, and A. M. Bayen, "Benchmarks for reinforcement learning in mixed-autonomy traffic," in *Proceedings of the 2nd Conference on Robot Learning*. PMLR, 2018, pp. 399–409.
- [5] A. R. Kreidieh, C. Wu, and A. M. Bayen, "Dissipating stop-and-go waves in closed and open networks via deep reinforcement learning," in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2018, pp. 1475–1480.
- [6] A. Authors, "Anonymous title," in *Anonymous Publication*, Anonymous Year.
- [7] B. D. Greenshields, J. R. Bibbins, W. Channing, and H. H. Miller, "A study of traffic capacity," in *Highway research board proceedings*, vol. 14. Washington, DC, 1935, pp. 448–477.
- [8] M. J. Lighthill and G. B. Whitham, "On kinematic waves ii. a theory of traffic flow on long crowded roads," *Proceedings of the royal society of london. series a. mathematical and physical sciences*, vol. 229, no. 1178, pp. 317–345, 1955.
- [9] P. I. Richards, "Shock waves on the highway," *Operations research*, vol. 4, no. 1, pp. 42–51, 1956.
- [10] R. E. Stern, S. Cui, M. L. Delle Monache, R. Bhadani, M. Bunting, M. Churchill, N. Hamilton, R. Haulcy, H. Pohlmann, F. Wu, B. Piccoli, B. Seibold, J. Sprinkle, and D. B. Work, "Dissipation of stop-and-go waves via control of autonomous vehicles: Field experiments," *Transportation Research Part C: Emerging Technologies*, vol. 89, pp. 205–221, 2018.
- [11] CIRCLES Consortium, "Traffic control via connected and automated vehicles: An open-road field experiment with 100 cavs," *arXiv preprint arXiv:2402.17043*, 2024.
- [12] E. D. Arnold, Jr., "Ramp metering: a review of the literature," Virginia Transportation Research Council, Tech. Rep., 1998.
- [13] M. Papageorgiou and A. Kotsialos, "Freeway ramp metering: An overview," *IEEE transactions on intelligent transportation systems*, vol. 3, no. 4, pp. 271–281, 2002.
- [14] NVIDIA Corporation, "NVIDIA jetson orin nano developer kit," <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/>, 2025, accessed: 2026-09-12.
- [15] Cambridge Mobile Telematics, "Cambridge mobile telematics," 2025, accessed: 2025-05-15. [Online]. Available: <https://www.cmtelematics.com/>
- [16] Samsara Inc., "Samsara: The connected operations cloud," 2025, accessed: 2025-05-15. [Online]. Available: <https://www.samsara.com/>
- [17] HERE Technologies, "HERE traffic API v7 developer guide," <https://www.here.com/docs/bundle/traffic-api-v7-api-reference>, 2025, accessed: 2026-09-12.
- [18] P. A. Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner, and E. Wießner, "Microscopic traffic simulation using SUMO," in *Proceedings of the IEEE Intelligent Transportation Systems Conference (ITSC)*, 2018, pp. 2575–2582.
- [19] D. Salles, S. Kaufmann, and H.-C. Reuss, "Extending the intelligent driver model in SUMO and verifying the drive off trajectories with aerial measurements," in *SUMO Conference Proceedings*, vol. 1, 2020, pp. 1–25.
- [20] M. Treiber, A. Hennecke, and D. Helbing, "Congested traffic states in empirical observations and microscopic simulations," *Physical Review E*, vol. 62, no. 2, pp. 1805–1824, 2000.
- [21] F. L. Hall and K. Agyemang-Duah, "Freeway capacity drop and the definition of capacity," *Transportation Research Record*, no. 1320, pp. 91–98, 1991.
- [22] Transport for New South Wales, *SCATS Traffic System: Sydney Co-ordinated Adaptive Traffic System*, <https://scats.transport.nsw.gov.au/>, 2024, accessed: 2024-10-21.
- [23] Transport Research Laboratory (TRL), *SCOOT: Split Cycle Offset Optimisation Technique*, <https://trl.co.uk/solutions/scoot>, 2024, accessed: 2024-10-21.
- [24] T. Seo, A. M. Bayen, T. Kusakabe, and Y. Asakura, "Traffic state estimation on highway: A comprehensive survey," *Annual Reviews in Control*, vol. 43, pp. 128–151, 2017.
- [25] J. C. Herrera, D. B. Work, R. Herring, X. Ban, Q. Jacobson, and A. M. Bayen, "Evaluation of traffic data obtained via GPS-enabled mobile phones: The Mobile Century field experiment," *Transportation Research Part C: Emerging Technologies*, vol. 18, no. 4, pp. 568–583, 2010.
- [26] SAE International, "V2X communications message set dictionary," SAE International, Tech. Rep. J2735, 2020.
- [27] ETSI, "Intelligent transport systems (ITS); vehicular communications; basic set of applications; part 2: Specification of cooperative awareness basic service," European Telecommunications Standards Institute, Tech. Rep. EN 302 637-2 V1.4.1, 2019.
- [28] J. B. Kenney, "Dedicated short-range communications (DSRC) standards in the united states," *Proceedings of the IEEE*, vol. 99, no. 7, pp. 1162–1182, 2011.
- [29] ETSI, "Intelligent transport systems (ITS); vehicular communications; basic set of applications; collective perception service; release 2," European Telecommunications Standards Institute, Tech. Rep. TS 103 324 V2.1.1, 2023.
- [30] Z. Yan, A. R. Kreidieh, E. Vinitzky, A. M. Bayen, and C. Wu, "Unified automatic control of vehicular systems with reinforcement learning," *IEEE Transactions on Automation Science and Engineering*, vol. 20, no. 2, p. 789–804, Apr. 2023. [Online]. Available: <http://dx.doi.org/10.1109/TASE.2022.3168621>
- [31] J.-H. Cho, S. Li, J. Kim, and C. Wu, "Temporal transfer learning for traffic optimization with coarse-grained advisory autonomy," *arXiv preprint arXiv:2312.09436*, 2023.
- [32] L. Fridman, "Human-centered autonomous vehicle systems: Principles of effective shared autonomy," *arXiv preprint arXiv:1810.01835*, 2018.
- [33] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [34] NVIDIA Corporation, *NVIDIA Jetson Platform Services Documentation*, 2023, accessed: 2025-08-22. [Online]. Available: <https://docs.nvidia.com/jetson/jps/index.html>
- [35] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, 2016, pp. 3464–3468.
- [36] L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, and H. Zhao, "Depth anything: Unleashing the power of large-scale unlabeled data," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 10 371–10 381.